ECE 4502/6502 & CS 6501 Machine Learning on Graphs: Homework 2

Due: 03/26/2025 11:59 PM EST

Note: The assignment must be submitted electronically on Canvas.

1. GNN Expressiveness (28 points)

Graph Neural Networks (GNNs) are a class of neural network architectures used for deep learning on graph-structured data. Broadly, GNNs aim to generate high-quality embeddings of nodes by iteratively aggregating feature information from local graph neighborhoods using neural networks. These embeddings can then be used for recommendations, classification, link prediction, or other downstream tasks. Two important types of GNNs are GCNs (Graph Convolutional Networks) and GraphSAGE (Graph Sampling and Aggregation).

Let $G = (V, E)$ denote a graph with node feature vectors X_u for $u \in V$. To generate the embedding for a node u, we use the neighborhood of the node as the computation graph. At every layer l, for each pair of nodes $u \in V$ and its neighbor $v \in V$, we compute a message function via neural networks, and apply a convolutional operation that aggregates the messages from the node's local graph neighborhood (Figure 1), and updates the node's representation at the next layer. By repeating this process through K GNN layers, we capture feature and structural information from a node's local K-hop neighborhood. For each of the message computation, aggregation, and update functions, the learnable parameters are shared across all nodes in the same layer.

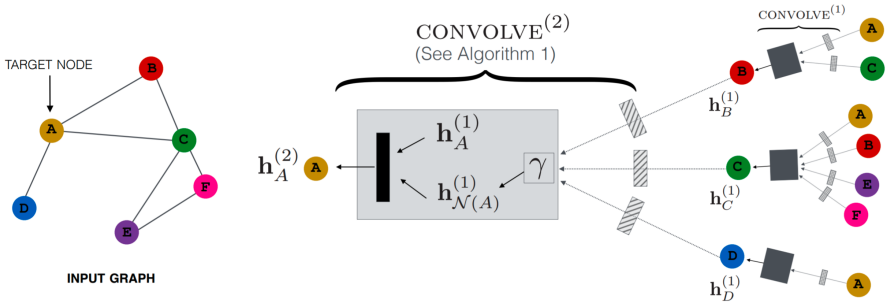


Figure 1: GNN architecture

We initialize the feature vector for node X_u based on its individual node attributes. If we already have outside information about the nodes, we can embed that as a feature vector. Otherwise, we can use a constant feature (e.g., a vector of ones) or the degree of the node as the feature vector.

These are the key steps in each layer of a GNN:

- **Message computation:** We use a neural network to learn a message function between nodes. For each pair of nodes u and its neighbor v, the neural network message function can be expressed as $M(h_u^k, h_v^k, e_{u,v})$. In GCN and GraphSAGE, this can simply be $\sigma(Wh_v + b)$, where W and b are the weights and bias of a neural network linear layer. Here h_u^k refers to the hidden representation of node u at layer k, and $e_{u,v}$ denotes available information about the edge (u, v), like the edge weight or other features. For GCN and GraphSAGE, the neighbors of u are simply defined as nodes that are connected to u. However, many other variants of GNNs have different definitions of neighborhood.

- **Aggregation:** At each layer, we apply a function to aggregate information from all of the neighbors of each node. The aggregation function is usually permutation invariant, to reflect the fact that nodes' neighbors have no canonical ordering. In a GCN, the aggregation is done by a weighted sum, where the weight for aggregating from v to u corresponds to the (u, v) entry of the normalized adjacency matrix $D^{-1/2}AD^{-1/2}$.

- **Update:** We update the representation of a node based on the aggregated representation of the neighborhood. For example, in GCNs, a multi-layer perceptron (MLP) is used; GraphSAGE combines a skip layer with the MLP.

- **Pooling:** The representation of an entire graph can be obtained by adding a pooling layer at the end. The simplest pooling methods are just taking the mean, max, or sum of all of the individual node representations. This is usually done for the purposes of graph classification.

We can formulate the Message computation, Aggregation, and Update steps for a GCN as a layer-wise propagation rule given by:

$$h^{k+1} = \sigma\left(D^{-1/2}AD^{-1/2}h^k W^k\right)$$

where h^k represents the matrix of activations in the k-th layer, $D^{-1/2}AD^{-1/2}$ is the normalized adjacency matrix of graph G, W^k is a layer-specific learnable matrix, and σ is a non-linearity function. Dropout and other forms of regularization can also be used.

We provide the pseudo-code for GraphSAGE embedding generation below.

In this question, we investigate the effect of the number of message passing layers on the expressive power of Graph Convolutional Networks. In neural networks, expressiveness refers to the set of functions (usually the loss function for classification or regression tasks) a neural network is able to compute, which depends on the structural properties of a neural network architecture.

Algorithm 1: Pseudo-code for forward propagation in GraphSAGE

Input : Graph $G(V, E)$; input features $\{x_v, \forall v \in V\}$; depth K; non-linearity σ; weight matrices $\{W^k, \forall k \in [1, K]\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^V$; aggregator functions $\{\text{AGGREGATE}_k, \forall k \in [1, K]\}$

Output: Vector representations z_v for all $v \in V$

$h_v^0 \leftarrow x_v, \forall v \in V$;
for $k = 1...K$ **do**
  for $v \in V$ **do**
    $h_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{h_u^{k-1}, \forall u \in \mathcal{N}(v)\})$ // aggregation
    $h_v^k \leftarrow \sigma\left(W^k \cdot \text{CONCAT}(h_v^{k-1}, h_{\mathcal{N}(v)}^k)\right)$ // MLP with skip connection
  $h_v^k \leftarrow h_v^k / \|h_v^k\|_2, \forall v \in V$ // update step
$z_v \leftarrow h_v^K, \forall v \in V$

1.1 Effect of Depth on Expressiveness (4 points)

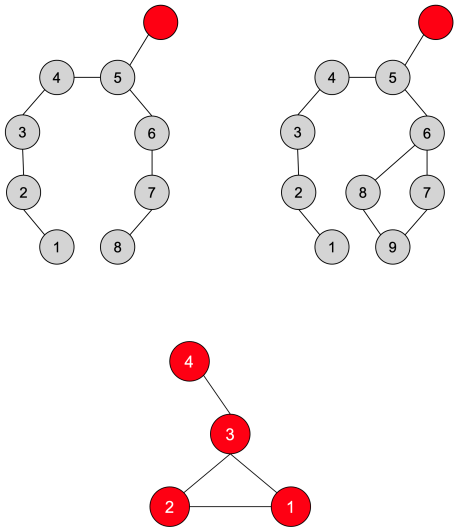
Consider the following 2 graphs, where all nodes have a 1-dimensional initial feature vector $x = [1]$. We use a simplified version of GNN, with no non-linearity, no learned linear transformation, and sum aggregation. Specifically, at every layer, the embedding of node v is updated as the sum over the embeddings of its neighbors $N(v)$ and its current embedding h_t^v to get h_{t+1}^v. We run the GNN to compute node embeddings for the 2 red nodes respectively. Note that the 2 red nodes have different 5-hop neighborhood structures (note this is not the minimum number of hops for which the neighborhood structure of the 2 nodes differs). How many layers of message passing are needed so that these 2 nodes can be distinguished (i.e., have different GNN embeddings)?

1.2 Random Walk Matrix (4 points)

Consider the graph shown below.

1. Assume that the current distribution over nodes is $r = [0, 0, 1, 0]$, and after the random walk, the distribution is $M \cdot r$. What is the random walk transition matrix M, where each row of M corresponds with the node ID in the graph?

2. What is the limiting distribution r, namely the eigenvector of M that has an eigenvalue of 1 ($r = Mr$)? Write your answer rounded to the nearest hundredth place and in the following form, e.g. $[1.00, 0.11, 0.46, 0.00]$. Note that before reporting, you should normalize r (L2-norm).



1.3 Relation to Random Walk (i) (4 points)

Let $h_i^{(l)}$ be the embedding of node i at layer l. Suppose that we are using a mean aggregator for message passing, and omit the learned linear transformation and non-linearity:

$$h_i^{(l+1)} = \frac{1}{|N_i|} \sum_{j \in N_i} h_j^{(l)}.$$

If we start at a node u and take a uniform random walk for 1 step, the expectation over the layer-l embeddings of nodes we can end up with is $h_u^{(l+1)}$, exactly the embedding of u in the next GNN layer.

What is the transition matrix of the random walk? Describe the transition matrix using the adjacency matrix A and degree matrix D, a diagonal matrix where $D_{i,i}$ is the degree of node i.

1.4 Relation to Random Walk (ii) (4 points)

Suppose that we add a skip connection to the aggregation from Question 1.3:

$$h_i^{(l+1)} = \frac{1}{2} h_i^{(l)} + \frac{1}{2} \frac{1}{|N_i|} \sum_{j \in N_i} h_j^{(l)}.$$

What is the new corresponding transition matrix?

1.5 Over-Smoothing Effect (5 points)

In Question 1.1, we see that increasing depth could give more expressive power. On the other hand, a very large depth also gives rise to the undesirable effect of over-smoothing. Assume we are still using the aggregation function from Question 1.3:

$$h_i^{(l+1)} = \frac{1}{|N_i|} \sum_{j \in N_i} h_j^{(l)}.$$

Show that the node embedding $h_i^{(l)}$ will converge as $l \to \infty$. Here we assume that the graph is connected and has no bipartite components.

Over-smoothing refers to the problem of node embeddings converging to the same value, making them indistinguishable and thus useless for downstream tasks. However, in practice, learnable weights, non-linearity, and other architecture choices can alleviate the over-smoothing effect.

Hint: Think about the Markov Convergence Theorem. Is the Markov chain irreducible and aperiodic? You don't need to be super rigorous with your proof.

1.6 Learning BFS with GNN (7 points)

Next, we investigate the expressive power of GNN for learning simple graph algorithms. Consider breadth-first search (BFS), where at every step, nodes that are connected to already visited nodes become visited.

Suppose that we use a GNN to learn to execute the BFS algorithm. Assume the embeddings are 1-dimensional. Initially, all nodes have an input feature of 0, except a source node which has an input feature of 1. At every step, nodes reached by BFS have an embedding of 1, and nodes not reached by BFS have an embedding of 0.

Describe a message function, an aggregation function, and an update rule for the GNN such that it learns the task perfectly.

2. Node Embedding and its Relation to Matrix Factorization (12 points)

Recall in lecture that matrix factorization and the encoder-decoder view of node embeddings are closely related. When properly formulating the encoder-decoder and the objective function, we can find equivalent matrix factorization approaches.

Note that in matrix factorization, we are optimizing for $L2$ distance; in encoder-decoder examples such as DeepWalk and node2vec, we often use log-likelihood as in lecture slides. The goal is to approximate A with $Z^T Z$, but for this question, stick with the $L2$ objective function.

2.1 Simple Matrix Factorization (3 points)

In simple matrix factorization, the objective is to approximate the adjacency matrix A by the product of the embedding matrix with its transpose. The optimization objective is:

$$\min_Z ||A - Z^T Z||_2.$$

In the encoder-decoder perspective of node embeddings, what is the decoder?

2.2 Alternate Matrix Factorization (3 points)

In linear algebra, we define a bilinear form as $z_i^T W z_j$, where W is a matrix. Suppose that we define the decoder as the bilinear form, what would be the objective function for the corresponding matrix factorization?

2.3 Relation to Eigen-decomposition (3 points)

Recall the eigen-decomposition of a matrix. What would be the condition of W such that the matrix factorization in 2.2 is equivalent to learning the eigen-decomposition of matrix A?

2.4 Multi-hop Node Similarity (3 points)

Define node similarity using the multi-hop definition: two nodes are similar if they are connected by at least one path of length at most k, where k is a parameter (e.g., $k = 2$).

Suppose that we use the same encoder (embedding lookup) and decoder (inner product) as before. What would be the corresponding matrix factorization problem we want to solve?

3. GCN (10 points)

Consider a graph $G = (V, E)$, with node features $x(v)$ for each $v \in V$. For each node $v \in V$, let $h_v^{(0)} = x(v)$ be the node's initial embedding. At each iteration k, the embeddings are updated as:

$$h_{N(v)}^{(k)} = \text{AGGREGATE}\left(\left\{h_u^{(k-1)}, \forall u \in N(v)\right\}\right)$$

$$h_v^{(k)} = \text{COMBINE}\left(h_v^{(k-1)}, h_{N(v)}^{(k)}\right),$$

where AGGREGATE and COMBINE are functions to be defined. Note that the argument to AGGREGATE, $\{h_u^{(k-1)}, \forall u \in N(v)\}$, is a multi-set. That is, since multiple nodes can have the same embedding, the same element can occur multiple times in the set.

Finally, a graph itself may be embedded by computing some function applied to the multi-set of all the node embeddings at some final iteration K, which we denote as:

$$\text{READOUT}\left(\left\{h_v^{(K)}, \forall v \in V\right\}\right).$$

We want to use the graph embeddings above to test whether two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic. Recall that this is true if and only if there is some bijection $\phi : V_1 \to V_2$ between nodes of G_1 and G_2 such that for any $u, v \in V_1$:

$$(u, v) \in E_1 \iff (\phi(u), \phi(v)) \in E_2.$$

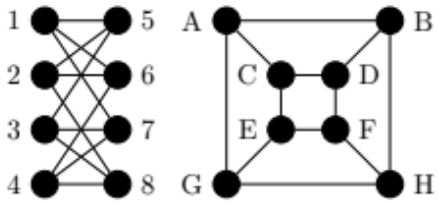
The way we use the model above to test isomorphism is the following: For the two graphs, if their READOUT functions differ, that is:

$$\text{READOUT}\left(\left\{h_v^{(K)}, \forall v \in V_1\right\}\right) \neq \text{READOUT}\left(\left\{h_v^{(K)}, \forall v \in V_2\right\}\right),$$

we conclude the graphs are not isomorphic. Otherwise, we conclude the graphs are isomorphic. Note that this algorithm is not perfect: graph isomorphism is thought to be hard! Below, we will explore the expressiveness of these graph embeddings.

3.1 Isomorphism Check (4 point)

Are the following two graphs isomorphic? If so, demonstrate an isomorphism between the sets of vertices. To demonstrate an isomorphism between two graphs, you need to find a one-to-one correspondence between their nodes and edges. If these two graphs are not isomorphic, prove it by finding a structure (node and/or edge) in one graph that is not present in the other.



3.2 Aggregation Choice (6 points)

The choice of the AGGREGATE function is important for the expressiveness of the model above. Three common choices are:

$$\text{AGGREGATE}_{\max}\left(\left\{h_u^{(k-1)}, \forall u \in N(v)\right\}\right)_i = \max_{u \in N(v)}\left(h_u^{(k-1)}\right)_i \quad \text{(element-wise max)}$$

$$\text{AGGREGATE}_{\text{mean}}\left(\left\{h_u^{(k-1)}, \forall u \in N(v)\right\}\right) = \frac{1}{|N(v)|}\sum_{u \in N(v)} h_u^{(k-1)}$$

$$\text{AGGREGATE}_{\text{sum}}\left(\left\{h_u^{(k-1)}, \forall u \in N(v)\right\}\right) = \sum_{u \in N(v)} h_u^{(k-1)}.$$

Give an example of two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ and their initial node features, such that for two nodes $v_1 \in V_1$ and $v_2 \in V_2$ with the same initial features $h_{v_1}^{(0)} = h_{v_2}^{(0)}$, the updated features $h_{v_1}^{(1)}$ and $h_{v_2}^{(1)}$ are equal if we use mean and max aggregation, but different if we use sum aggregation.

Hint: Your node features can be scalars rather than vectors, i.e., one-dimensional node features instead of n-dimensional. You are free to arbitrarily choose the number of nodes and their connections (i.e., edges between nodes) in your example.